Apache funcionando en Docker - Primera Prueba

Antes de empezar con el código del juego de damas en PHP, lo primero que hice fue montar un servidor Apache básico usando Docker en la instancia de AWS. Esta prueba es lo que llamamos en ingeniería una "Validación de Infraestructura".

El objetivo era simple: confirmar que la base del proyecto (la red de AWS y el motor de Docker) funcionaba perfectamente antes de complicarlo todo con el código. Si algo fallaba en este punto, sabría que el problema era de red o de configuración del servidor, y no de un error en mi lógica de programación.

Estrategia de Despliegue - ¿Por qué hacerlo así?

Puede parecer que desplegar un servidor "vacío" es un paso innecesario, pero lo hice por tres motivos estratégicos que garantizan la estabilidad del TFG:

1. Validación de la red de AWS: Necesitaba confirmar que el Security Group de la instancia permitía el tráfico por el puerto 80 y que podía acceder desde internet usando la IP pública. Si la red no respondía aquí, daba igual lo bien que estuviera escrito el código después.
2. Aislamiento de fallos: Al usar la imagen oficial de Apache (httpd:latest), me aseguro de que el entorno es 100% estable. Si algo no cargaba, el problema estaba en la conexión o en Docker, no en mi código PHP.
3. Referencia de consumo : Quería ver cuántos recursos gasta un servidor en reposo. Esto me sirve para comparar más adelante cuánto sube el consumo de RAM y CPU cuando ya esté el juego funcionando con toda su lógica.

Implementación

Para que el despliegue fuera limpio y fácil de repetir, utilicé Docker Compose. En lugar de lanzar comandos largos, definí todo en un archivo docker-compose.yml que me permite levantar el servicio en segundos.

Estructura del despliegue

Utilicé un mapeo de volúmenes para conectar mi carpeta local /html/ con la ruta de Apache dentro del contenedor. Así, cualquier cambio que haga en mis archivos se ve reflejado al instante en el navegador.

```
version: '3.8'
services:
  web:
    image: httpd:latest
    ports:
```

```
  - "80:80"
volumes:
  - ./html:/usr/local/apache2/htdocs/
restart: always
```

Test de contenido

Cree un archivo index.html básico con el título "Sede Nube - TFG" para verificar que Apache servía los archivos correctamente desde el volumen montado.

```
  GNU nano 7.2                                  /home/ubuntu/mi_proyecto_web/html/index.html
<!DOCTYPE html>
<html>
<head>
    <title>Sede Nube - TFG</title>
</head>
<body style="text-align: center; font-family: Arial; margin-top: 50px;">
    <h1 style="color: #FF9900;">¡Hola desde la Sede Nube de AWS! ☁️</h1>
    <p>El servidor Apache en Docker está funcionando correctamente.</p>
    <p>¡Lista para recibir los archivos de Carlitos por la VPN!</p>
</body>
</html>
```

Resultados de la Prueba

Una vez lanzado el comando sudo docker-compose up -d, los datos reales del sistema confirmaron que la elección de la instancia t3.micro fue acertada:

- Estado: Funcionando a la primera. El contenedor se levantó sin errores y mapeó el puerto 80 correctamente.
- Consumo de Memoria: Solo utilizó 14.0 MiB de RAM. Esto es poquísimo (un 1.54% de la capacidad de la instancia), lo que nos deja mucho margen para cuando metamos PHP y la base de datos.
- CPU: Se mantuvo en un 0.00% en reposo, validando que la carga de un servidor web bajo Docker es mínima.
- Conectividad: La página de prueba cargó perfectamente desde la IP pública del servidor web (10.1.0.85).

```
ubuntu@ip-10-1-0-85:~$ sudo docker stats mi_proyecto_web_web_1 --no-stream
CONTAINER ID   NAME                    CPU %     MEM USAGE / LIMIT   MEM %     NET I/O           BLOCK I/O         PIDS
5783e3d1e9df   mi_proyecto_web_web_1   0.00%     14MiB / 911.4MiB    1.54%     15.5kB / 26.1kB   7.52MB / 4.1kB    82
```

Conclusión

Con esta prueba de concepto he demostrado que los "cimientos" del TFG son sólidos. La comunicación entre la red de AWS y Docker es perfecta y el servidor es extremadamente ligero.

Al tener esta base segura y saber que no hay problemas de red "escondidos", ya estoy listo para pasar a la siguiente fase: actualizar el contenedor para que soporte PHP 8.2 y empezar a subir los archivos del juego de damas.